

The Complete Preparation Guide

Odoo Coderbyte Technical Assessment

*For Python Developers — String Manipulation, SQL, Graphs, Stacks, Arrays
& Math*

June 2026 Edition

Table of Contents

Chapter 1: Understanding the Assessment

Chapter 2: String Manipulation Mastery

Chapter 3: SQL & Database Design

Chapter 4: Graph Algorithms — BFS Shortest Path

Chapter 5: Stack & Queue Deep Dive

Chapter 6: Arrays & Two Pointers

Chapter 7: Math & Logic

Chapter 8: Exam Strategy & Platform Tips

Appendix: Quick Reference Cheat Sheet

Chapter 1: Understanding the Assessment

The Odoo Coderbyte assessment is a **general coding proficiency test** — not an Odoo framework quiz. You will not be asked about ORM, XML views, QWeb, or Odoo-specific APIs. The goal is to filter candidates who can write clean, correct code under time pressure.

What to Expect

- **Platform:** Coderbyte (browser-based, webcam proctored, session recorded)
- **Duration:** 1 hour for your specific invite
- **Languages:** Python and Java (Python is strongly recommended for readability and speed)
- **Question count:** Typically 2–3 coding problems
- **Difficulty:** ~67% Easy, ~33% Medium; Hard problems are rare
- **Typical mix:** 1 String/DSA + 1 SQL/Database design + optionally 1 Graph or Stack problem

Proctoring Reality

Coderbyte monitors *document.visibilitychange* and *window.onblur* to detect tab switching and window focus loss. It also records your webcam session. Human reviewers inspect flagged recordings. The next interview round requires you to **explain and optimize your solutions** — so you must fully understand every line you submit.

NOTE: The technical interview after this round will ask you to walk through your Coderbyte code. If you cannot explain your approach, you will be disqualified regardless of whether your code passed.

Pipeline Overview

1. **Coderbyte Assessment** (coding + proctoring)
2. **Aptitude & English Test** (MCQs + email writing)
3. **Technical Interview** (explain solutions, live coding, OOP)
4. **HR / Final Round**

Chapter 2: String Manipulation Mastery

String manipulation is the **most frequently reported topic** on the Odoo assessment. Past candidates mention problems involving parsing, validation, transformation, run-length encoding, wildcard matching, and HTML tag matching. Master this chapter first.

2.1 Python String Essentials

Python strings are immutable. Every "modification" creates a new string. For frequent concatenations, collect pieces in a list and use *join()* to stay $O(n)$.

```
# Essential methods – drill until they are muscle memory
s = " Hello, World! "

s.strip()          # "Hello, World!" – remove outer whitespace
s.lower()          # " hello, world! "
s.upper()          # " HELLO, WORLD! "
s.split(",")       # [" Hello", " World! "]
"-".join(["a","b"]) # "a-b"
s.replace("l", "x") # " Hexxo, Worxd! "
s.find("World")     # 9 (index or -1)
s.count("l")        # 3
s.startswith("He")  # False (leading spaces!)
s[::-1]            # "!dlrow ,olleH " – reverse

# Character class checks
"5".isdigit()      # True
"a".isalpha()      # True
" ".isspace()      # True
"_" .isalnum()     # False
```

TIP: In a timed test, you cannot afford to look up methods. Write these on a physical cheat sheet and place it beside your screen if allowed, or memorize them through repetition.

2.2 Two-Pointer Technique

Place one pointer at each end and move them toward the center. This avoids extra data structures and keeps space at $O(1)$. It is the standard solution for palindromes, reverse string, and two-sum (sorted).

```
def is_palindrome(s: str) -> bool:
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True

# Time:  $O(n)$ , Space:  $O(1)$ 
# Faster than s == s[::-1] for very long strings and shows
# algorithmic thinking in follow-up interviews.
```

2.3 Parsing & Validation

When validating a string (phone numbers, IDs, custom formats), follow this systematic approach: (1) Sanitize by removing decorative chars. (2) Check length and character sets. (3) Use a state machine for complex grammars.

```
def validate_id(code: str) -> bool:
    # Example: ID must be 2 letters + 4 digits
    code = code.strip()
    if len(code) != 6:
        return False
    return code[:2].isalpha() and code[2:].isdigit()

def clean_phone(phone: str) -> str:
    # Remove common decorative characters
    return phone.replace("-", "").replace(" ", "").replace("(", "").replace(")", "")
```

2.4 Run-Length Encoding (RLE)

RLE compresses a string by replacing consecutive identical characters with the character followed by the count. This tests loop control and the critical edge case of appending the final run after the loop ends.

```
def run_length_encode(s: str) -> str:
    if not s:
        return ""

    result = []
    count = 1

    for i in range(1, len(s)):
        if s[i] == s[i-1]:
            count += 1
        else:
            result.append(s[i-1] + str(count))
            count = 1

    # CRITICAL: append the final run
    result.append(s[-1] + str(count))
    return "".join(result)

# Example: "aaabbc" -> "a3b2c1"
# Edge case: "a" -> "a1"
```

TIP: The most common RLE bug is forgetting to append the final run after the loop. Always verify your code with a single-character string.

2.5 Simplified Wildcard Matching

A reported topic. "?" matches any single character; "*" matches any sequence (including empty). For Easy problems, a greedy two-pointer approach often suffices. Understand the backtracking logic: when "*" is encountered, record its position and the current string index, then advance. If a mismatch occurs later, backtrack to let "*" consume one more character.

```
def is_match(s: str, p: str) -> bool:
    s_idx = p_idx = 0
    star_idx = s_tmp_idx = -1

    while s_idx < len(s):
```

```

if p_idx < len(p) and p[p_idx] in {s[s_idx], '?'}:
    s_idx += 1
    p_idx += 1
elif p_idx < len(p) and p[p_idx] == '*':
    star_idx = p_idx
    s_tmp_idx = s_idx
    p_idx += 1
elif star_idx == -1:
    return False
else:
    # Backtrack: let '*' consume one more char
    p_idx = star_idx + 1
    s_tmp_idx += 1
    s_idx = s_tmp_idx

# Remaining pattern must be all '*'
return all(c == '*' for c in p[p_idx:])

# Time: O(n), Space: O(1)

```

2.6 HTML Tag Matching with a Stack

A reported Odoos problem involves checking if HTML tags are properly nested. This combines string parsing with stack logic. Every opening tag is pushed; every closing tag must match the top of the stack.

```
def validate_html_tags(html: str) -> bool:
    stack = []
    i = 0
    while i < len(html):
        if html[i] == '<':
            j = html.find('>', i)
            if j == -1:
                return False # Unclosed tag

            tag = html[i+1:j]
            if tag.startswith('/'):
                # Closing tag
                if not stack or stack[-1] != tag[1:]:
                    return False
                stack.pop()
            else:
                # Opening tag – ignore attributes
                tag_name = tag.split()[0]
                stack.append(tag_name)
            i = j
        i += 1
    return len(stack) == 0

# Example: "<div><p></p></div>" -> True
# Example: "<div><p></div></p>" -> False
```

NOTE: In the actual Coderbyte variant, the logic may be more nuanced — e.g., determining if changing ONE element could fix the structure. Understand the core stack principle first, then adapt to the specific prompt.

PRACTICE: Valid Palindrome (Alphanumeric Only)

Given a string, determine if it is a palindrome considering only alphanumeric characters and ignoring cases. Example: "A man, a plan, a canal: Panama" returns True.

```
def is_palindrome(s: str) -> bool:
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True

# Time: O(n), Space: O(1)
```

PRACTICE: String Compression (RLE with Length Check)

Perform basic string compression using counts of repeated characters. If the compressed

string is not smaller than the original, return the original string.

```
def compress(s: str) -> str:
    if not s:
        return s

    compressed = []
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i-1]:
            count += 1
        else:
            compressed.append(s[i-1] + str(count))
            count = 1
    compressed.append(s[-1] + str(count))

    result = "".join(compressed)
    return result if len(result) < len(s) else s

# "aabcccccaaa" -> "a2b1c5a3"
# "abc" -> "abc" (compressed is longer)
```

PRACTICE: First Non-Repeating Character

Find the first non-repeating character in a string and return its index. If it does not exist, return -1.

```
def first_uniq_char(s: str) -> int:
    from collections import Counter
    count = Counter(s)
    for i, ch in enumerate(s):
        if count[ch] == 1:
            return i
    return -1

# Time: O(n), Space: O(1) [at most 26/128 characters]
```


Chapter 3: SQL & Database Design

The second most common question type is SQL or database schema design. You may be asked to write a query with JOIN, GROUP BY, and aggregation, or to design a schema for a simple system (course platform, booking system, animal care center).

3.1 Schema Design Principles

When designing a database from a description, follow this checklist:

1. **Identify entities** — nouns in the problem (User, Course, Booking, Animal).
2. **Define primary keys** — usually an auto-increment integer ID.
3. **Define relationships** — one-to-one, one-to-many, many-to-many. Use foreign keys for one-to-many; use junction tables for many-to-many.
4. **Add constraints** — NOT NULL, UNIQUE, CHECK, DEFAULT.
5. **Consider indexing** — mention foreign keys and frequently searched columns.

Example: Course Platform

Design tables for students, courses, and enrollments. A student can enroll in many courses; a course has many students. This is a classic many-to-many relationship.

```
-- Students table
CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    email VARCHAR(255) UNIQUE NOT NULL,
    name VARCHAR(100) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Courses table
CREATE TABLE courses (
    course_id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(200) NOT NULL,
    instructor VARCHAR(100),
    credits INT CHECK (credits > 0)
);

-- Junction table for many-to-many
CREATE TABLE enrollments (
    enrollment_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT NOT NULL,
    course_id INT NOT NULL,
    enrolled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id),
    UNIQUE(student_id, course_id) -- prevent duplicate enrollments
);
```

TIP: In a design question, write CREATE TABLE statements. Even if the platform only asks for a description, providing SQL shows precision. Always mention foreign keys and junction tables.

3.2 Essential SQL Patterns

These patterns solve 90% of assessment SQL problems:

```
-- PATTERN 1: Aggregation with GROUP BY
SELECT
    course_id,
    COUNT(*) AS student_count,
    AVG(grade) AS avg_grade
FROM enrollments
WHERE enrolled_at >= '2025-01-01'
GROUP BY course_id
HAVING COUNT(*) > 5
ORDER BY student_count DESC;

-- PATTERN 2: INNER JOIN to combine entities
SELECT
    s.name,
    c.title
FROM students s
INNER JOIN enrollments e ON s.student_id = e.student_id
INNER JOIN courses c ON e.course_id = c.course_id
WHERE c.title LIKE '%Python%';

-- PATTERN 3: LEFT JOIN to find missing relationships
SELECT s.name
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
WHERE e.enrollment_id IS NULL; -- students with no enrollments

-- PATTERN 4: Subquery in WHERE
SELECT name FROM students
WHERE student_id IN (
    SELECT student_id FROM enrollments
    GROUP BY student_id
    HAVING COUNT(*) > 3
);
```

NOTE: The execution order in SQL is: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY. Understanding this helps you debug why a column alias cannot be used in WHERE.

3.3 Writing Queries Under Pressure

If the question asks for "the most popular course" or "the user with highest bookings", always think: (1) GROUP BY the entity, (2) aggregate with COUNT/SUM/MAX, (3) ORDER BY the aggregate DESC, (4) LIMIT 1. If ties are possible, use a subquery or window function (RANK).

```
-- Top N with ties (using subquery)
SELECT course_id, student_count
FROM (
    SELECT
        course_id,
        COUNT(*) AS student_count,
        RANK() OVER (ORDER BY COUNT(*) DESC) AS rnk
    FROM enrollments
    GROUP BY course_id
) ranked
WHERE rnk = 1;
```

PRACTICE: Design an Animal Care Center Schema

Track animals, their owners, veterinarians, and appointments. An animal has one owner. A veterinarian sees many animals. An animal has many appointments.

```
CREATE TABLE owners (  
    owner_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    phone VARCHAR(20)  
);  
  
CREATE TABLE veterinarians (  
    vet_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    specialization VARCHAR(50)  
);  
  
CREATE TABLE animals (  
    animal_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    species VARCHAR(50),  
    owner_id INT,  
    FOREIGN KEY (owner_id) REFERENCES owners(owner_id)  
);  
  
CREATE TABLE appointments (  
    appointment_id INT PRIMARY KEY AUTO_INCREMENT,  
    animal_id INT NOT NULL,  
    vet_id INT NOT NULL,  
    appointment_date DATETIME NOT NULL,  
    notes TEXT,  
    FOREIGN KEY (animal_id) REFERENCES animals(animal_id),  
    FOREIGN KEY (vet_id) REFERENCES veterinarians(vet_id)  
);
```

PRACTICE: SQL: Find Duplicate Emails

Write a SQL query to find all duplicate email addresses in a table named Person with columns id and email.

```
SELECT email  
FROM Person  
GROUP BY email  
HAVING COUNT(*) > 1;
```

PRACTICE: SQL: Department Top 3 Salaries

The Employee table has id, name, salary, departmentId. The Department table has id, name. Find the top 3 unique salaries in each department.

```
SELECT d.name AS Department, e.name AS Employee, e.salary AS Salary  
FROM (  
    SELECT *,  
        DENSE_RANK() OVER (  
            PARTITION BY departmentId  
            ORDER BY salary DESC  
        ) AS rnk  
    FROM Employee  
    ) e  
JOIN Department d ON e.departmentId = d.id
```

```
WHERE e.rnk <= 3;
```

Chapter 4: Graph Algorithms — BFS

Shortest Path

Graph problems appear on Odoo assessments, typically in the form of finding the shortest path between two nodes. Because Coderybyte problems often use unweighted edges (or implicit weight of 1), **Breadth-First Search (BFS)** is the optimal algorithm. It finds the shortest path in $O(V + E)$ time.

4.1 Graph Representation

For coding assessments, the adjacency list is the most practical representation. It is space-efficient and easy to traverse. You will typically receive edges as a list of pairs `[[0,1], [0,2], [1,3]]` or a dictionary mapping nodes to neighbors.

```
# Build adjacency list from edge list
edges = [[0, 1], [0, 2], [1, 3], [2, 3], [3, 4]]
graph = {}
for u, v in edges:
    graph.setdefault(u, []).append(v)
    graph.setdefault(v, []).append(u) # undirected

# Result: {0: [1,2], 1: [0,3], 2: [0,3], 3: [1,2,4], 4: [3]}
```

TIP: Always use `setdefault()` or `collections.defaultdict(list)` to avoid `KeyError` when a node has no outgoing edges yet.

4.2 BFS Algorithm Step-by-Step

BFS explores nodes layer by layer. Starting from the source node, it visits all direct neighbors (distance 1), then all neighbors of those neighbors (distance 2), and so on. The first time we reach the target node, we have found the shortest path.

```
from collections import deque

def shortest_path(graph, start, end):
    if start == end:
        return 0

    visited = set([start])
    queue = deque([(start, 0)]) # (node, distance)

    while queue:
        node, dist = queue.popleft()

        for neighbor in graph.get(node, []):
            if neighbor == end:
                return dist + 1
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, dist + 1))

    return -1 # No path exists

# Time: O(V + E), Space: O(V)
```

4.3 Why BFS and Not DFS?

DFS explores as deep as possible before backtracking. It can find a path, but not necessarily the shortest one. BFS guarantees the shortest path in unweighted graphs because it explores all nodes at distance d before any node at distance $d+1$.

```
# If you need the actual path, not just distance:
def shortest_path_with_route(graph, start, end):
    if start == end:
        return [start]

    visited = {start}
    queue = deque([(start, [start])])

    while queue:
        node, path = queue.popleft()

        for neighbor in graph.get(node, []):
            if neighbor == end:
                return path + [neighbor]
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))

    return []

# Warning: storing full paths uses more memory.
# Only use this if the problem explicitly asks for the path.
```

PRACTICE: Shortest Path in Binary Matrix

Given an $n \times n$ binary matrix grid, find the length of the shortest clear path from top-left to bottom-right. A clear path consists of cells with value 0, and you can move in 8 directions. If no path, return -1.

```
from collections import deque

def shortest_path_binary_matrix(grid):
    n = len(grid)
    if grid[0][0] == 1 or grid[n-1][n-1] == 1:
        return -1

    directions = [(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1, 1)]
    queue = deque([(0, 0, 1)]) # (row, col, distance)
    grid[0][0] = 1 # mark visited

    while queue:
        r, c, dist = queue.popleft()
        if r == n-1 and c == n-1:
            return dist

        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < n and grid[nr][nc] == 0:
                grid[nr][nc] = 1 # mark visited
                queue.append((nr, nc, dist + 1))

    return -1

# Time:  $O(n^2)$ , Space:  $O(n^2)$ 
```

PRACTICE: Rotting Oranges (Multi-Source BFS)

Given a grid where 0=empty, 1=fresh orange, 2=rotten orange, return the minimum minutes until no fresh orange remains. A rotten orange rots adjacent fresh oranges every minute.

```
from collections import deque

def oranges_rotting(grid):
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh = 0

    # Initialize: all rotten oranges are BFS sources
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                queue.append((r, c, 0))
            elif grid[r][c] == 1:
                fresh += 1

    minutes = 0
    directions = [(0,1),(0,-1),(1,0),(-1,0)]

    while queue:
        r, c, minutes = queue.popleft()
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                grid[nr][nc] = 2
                fresh -= 1
                queue.append((nr, nc, minutes + 1))

    return minutes if fresh == 0 else -1
```

Chapter 5: Stack & Queue Deep Dive

Stack problems are common on the Odoo assessment, especially for validating nested structures (parentheses, HTML tags) and evaluating expressions. The Stack is LIFO (Last-In-First-Out); the Queue is FIFO (First-In-First-Out). Python lists work as stacks via `append()/pop()`, but for queues always use `collections.deque` for $O(1)$ pops from the front.

5.1 Stack Fundamentals

A stack supports two core operations: push (add to top) and pop (remove from top). In Python, use a list and treat the end as the top. For peeking at the top element, access `stack[-1]`. Always check `len(stack) > 0` before popping to avoid `IndexError`.

```
stack = []
stack.append(10)    # push
stack.append(20)
top = stack[-1]     # peek -> 20
stack.pop()         # pop -> 20
size = len(stack)   # 1
is_empty = len(stack) == 0
```

5.2 Balanced Parentheses / Brackets

The classic stack problem. Every opening bracket must be closed in the correct order. Push opening brackets; when encountering a closing bracket, check if it matches the top of the stack.

```
def is_valid(s: str) -> bool:
    pair = {')': '(', ']': '[', '}': '{'}
    stack = []

    for ch in s:
        if ch in '([{':
            stack.append(ch)
        else:
            if not stack or stack[-1] != pair[ch]:
                return False
            stack.pop()

    return len(stack) == 0

# Time: O(n), Space: O(n)
# Edge cases: "(" -> False, ")" -> False, "" -> True
```

TIP: Use a dictionary to map closing brackets to their opening counterparts. This is cleaner than multiple if-statements and easier to extend.

5.3 MinStack Design

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time. The trick is to store tuples of (value, current_min) so that each element carries the minimum of all elements below it.


```

class MinStack:
    def __init__(self):
        self.stack = [] # each element is (value, minsofar)

    def push(self, val: int) -> None:
        if not self.stack:
            self.stack.append((val, val))
        else:
            current_min = min(val, self.stack[-1][1])
            self.stack.append((val, current_min))

    def pop(self) -> None:
        self.stack.pop()

    def top(self) -> int:
        return self.stack[-1][0]

    def getMin(self) -> int:
        return self.stack[-1][1]

# All operations are O(1).
# Alternative: use two stacks (main stack + min stack).

```

5.4 Evaluate Reverse Polish Notation (RPN)

RPN (postfix notation) evaluates without parentheses. When encountering a number, push it. When encountering an operator, pop the top two numbers, apply the operator, and push the result. The final stack element is the answer.

```

def eval_rpn(tokens: list[str]) -> int:
    stack = []
    ops = {'+', '-', '*', '/'}

    for token in tokens:
        if token not in ops:
            stack.append(int(token))
        else:
            b = stack.pop()
            a = stack.pop()
            if token == '+':
                stack.append(a + b)
            elif token == '-':
                stack.append(a - b)
            elif token == '*':
                stack.append(a * b)
            else:
                # Integer division truncating toward zero
                stack.append(int(a / b))

    return stack[0]

# Example: ["2","1","+","3","*"] -> ((2+1)*3) = 9
# Division trap: Python's // truncates toward negative infinity.
# Use int(a / b) to truncate toward zero.

```

NOTE: The division trap is critical. In Python, `-3 // 2` equals `-2`, but the problem expects `-1`. Always use `int(a / b)` for RPN problems, not `a // b`.

PRACTICE: Daily Temperatures

Given an array of daily temperatures, return an array where each element is the number of days until a warmer temperature. If no future day is warmer, put 0.

```
def daily_temperatures(temps: list[int]) -> list[int]:
    n = len(temps)
    result = [0] * n
    stack = [] # stores indices of decreasing temperatures

    for i in range(n):
        while stack and temps[i] > temps[stack[-1]]:
            prev_idx = stack.pop()
            result[prev_idx] = i - prev_idx
        stack.append(i)

    return result

# Time: O(n), Space: O(n)
# Monotonic stack: maintains decreasing order.
```

PRACTICE: Simplify Path (Unix-style)

Given a string path representing an absolute Unix path, simplify it. "." means parent directory, "." means current directory, multiple slashes should be treated as single slash.

```
def simplify_path(path: str) -> str:
    stack = []
    parts = path.split('/')

    for part in parts:
        if part == '' or part == '.':
            continue
        elif part == '..':
            if stack:
                stack.pop()
        else:
            stack.append(part)

    return '/' + '/'.join(stack)

# Example: "/home//foo/./bar/../baz" -> "/home/foo/baz"
```

Chapter 6: Arrays & Two Pointers

Array problems on the Odo assessment are typically Easy to Easy-Medium. The two-pointer technique is the most important pattern: it reduces time complexity from $O(n^2)$ to $O(n)$ and space complexity to $O(1)$.

6.1 Two-Pointer Technique

There are two variants: (1) **Opposite pointers** — one at start, one at end, moving toward center (palindromes, two-sum sorted). (2) **Same-direction pointers** — both start at beginning, one moves faster (sliding window, remove duplicates).

```
# Variant 1: Opposite pointers (Two Sum in sorted array)
def two_sum_sorted(nums: list[int], target: int) -> list[int]:
    left, right = 0, len(nums) - 1
    while left < right:
        current = nums[left] + nums[right]
        if current == target:
            return [left, right]
        elif current < target:
            left += 1
        else:
            right -= 1
    return []

# Time: O(n), Space: O(1)
# This only works because the array is sorted.

# Variant 2: Same-direction (Remove duplicates from sorted array)
def remove_duplicates(nums: list[int]) -> int:
    if not nums:
        return 0

    write = 1 # pointer for next unique element
    for read in range(1, len(nums)):
        if nums[read] != nums[read - 1]:
            nums[write] = nums[read]
            write += 1

    return write # new length

# Time: O(n), Space: O(1) – modifies array in-place
```

TIP: When the problem says "in-place" or "do not allocate extra space," think two pointers immediately. The write pointer overwrites the array while the read pointer scans.

6.2 Sliding Window (Fixed & Variable)

Sliding window is a variation of same-direction pointers. It maintains a subarray (window) that satisfies a condition and slides it across the array. Fixed-size windows are easier; variable-size windows require careful expansion and contraction.

```
# Fixed-size window: maximum sum of k consecutive elements
def max_sum_k(nums: list[int], k: int) -> int:
    if not nums or k > len(nums):
```

```

        return 0

    window_sum = sum(nums[:k])
    max_sum = window_sum

    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k]
        max_sum = max(max_sum, window_sum)

    return max_sum

# Time: O(n), Space: O(1)

```

PRACTICE: Move Zeroes

Given an array `nums`, move all 0s to the end while maintaining the relative order of non-zero elements. Do this in-place without making a copy of the array.

```

def move_zeroes(nums: list[int]) -> None:
    write = 0
    for read in range(len(nums)):
        if nums[read] != 0:
            nums[write] = nums[read]
            write += 1

    # Fill remaining positions with zeros
    for i in range(write, len(nums)):
        nums[i] = 0

# Time: O(n), Space: O(1)
# This is the same-direction two-pointer pattern.

```

PRACTICE: Container With Most Water

Given `n` non-negative integers representing vertical lines where the width of each line is 1, find two lines that together with the x-axis form a container that holds the most water.

```

def max_area(height: list[int]) -> int:
    left, right = 0, len(height) - 1
    max_water = 0

    while left < right:
        width = right - left
        current = min(height[left], height[right]) * width
        max_water = max(max_water, current)

        # Move the pointer at the shorter line
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1

    return max_water

# Time: O(n), Space: O(1)
# Greedy logic: the shorter line limits the area, so moving it
# is the only way to potentially find a larger area.

```

Chapter 7: Math & Logic

Math problems on the Odoo assessment are typically straightforward: reverse digits, basic computations, prime checks, or digit manipulation. They test attention to edge cases (negative numbers, overflow, zero) rather than advanced mathematics.

7.1 Reverse Digits

Reverse the digits of a 32-bit signed integer. If reversing causes overflow (outside $[-2^{31}, 2^{31}-1]$), return 0. The key is to pop digits from the end using modulo and push them to the result using multiplication.

```
def reverse(x: int) -> int:
    INT_MIN, INT_MAX = -2**31, 2**31 - 1
    result = 0

    while x != 0:
        # Python's % behaves differently for negatives, so we use
        # a trick to match C/Java behavior
        digit = int(math.fmod(x, 10))
        x = int(x / 10)

        # Check overflow before pushing
        if (result > INT_MAX // 10 or
            (result == INT_MAX // 10 and digit > 7)):
            return 0
        if (result < INT_MIN // 10 or
            (result == INT_MIN // 10 and digit < -8)):
            return 0

        result = result * 10 + digit

    return result

# Alternative for Python (simpler, but know the math version):
def reverse_simple(x: int) -> int:
    sign = -1 if x < 0 else 1
    result = sign * int(str(abs(x))[::-1])
    return result if -2**31 <= result <= 2**31 - 1 else 0
```

TIP: For interviews, know the mathematical version (modulo and division). The string conversion version is acceptable for speed in a timed test, but the interviewer may ask for the math approach in the next round.

7.2 Prime Number Check

Check if a number is prime. The naive $O(n)$ approach is too slow for large numbers. Instead, check divisibility up to $\sqrt{n}$. Also handle edge cases: numbers less than 2 are not prime; 2 is the only even prime.

```
import math

def is_prime(n: int) -> bool:
    if n < 2:
        return False
```

```

    if n == 2:
        return True
    if n % 2 == 0:
        return False

    for i in range(3, int(math.sqrt(n)) + 1, 2):
        if n % i == 0:
            return False
    return True

# Time: O(sqrt(n)), Space: O(1)
# Checking only odd numbers after 2 cuts the work in half.

```

7.3 GCD and LCM

The Euclidean algorithm computes GCD efficiently: $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$. LCM follows from GCD: $\text{lcm}(a, b) = \text{abs}(a*b) // \text{gcd}(a, b)$.

```

import math

def gcd(a: int, b: int) -> int:
    while b:
        a, b = b, a % b
    return a

def lcm(a: int, b: int) -> int:
    return abs(a * b) // gcd(a, b)

# Python also has math.gcd() built-in.

```

PRACTICE: Happy Number

Write an algorithm to determine if a number n is happy. A happy number is defined by repeatedly replacing the number with the sum of the squares of its digits, eventually reaching 1. If it loops endlessly in a cycle, it is not happy.

```

def is_happy(n: int) -> bool:
    def get_next(num):
        total = 0
        while num > 0:
            digit = num % 10
            total += digit * digit
            num //= 10
        return total

    slow = n
    fast = get_next(n)

    while fast != 1 and slow != fast:
        slow = get_next(slow)
        fast = get_next(get_next(fast))

    return fast == 1

# Floyd's Cycle Detection (tortoise and hare).
# Time: O(log n), Space: O(1)

```

PRACTICE: Count Primes (Sieve of Eratosthenes)

Count the number of prime numbers less than a non-negative number n .

```
def count_primes(n: int) -> int:
    if n <= 2:
        return 0

    is_prime = [True] * n
    is_prime[0] = is_prime[1] = False

    for i in range(2, int(n**0.5) + 1):
        if is_prime[i]:
            for j in range(i*i, n, i):
                is_prime[j] = False

    return sum(is_prime)

# Time:  $O(n \log \log n)$ , Space:  $O(n)$ 
# Start inner loop at  $i*i$  because smaller multiples are already marked.
```

Chapter 8: Exam Strategy & Platform Tips

You have 1 hour and 2–3 problems. Time management and psychological calm are as important as algorithmic knowledge. This chapter covers how to behave during the test and how to prepare in the final minutes.

8.1 Time Allocation (1 Hour)

- **0:00–0:05:** Read all problems. Pick the easiest one first.
- **0:05–0:25:** Solve Problem 1 completely. Write clean code, add comments, test mentally.
- **0:25–0:45:** Solve Problem 2. If stuck after 10 minutes, write a brute-force solution first, then optimize.
- **0:45–0:55:** Solve Problem 3 (if present) or revisit Problem 1/2 for edge cases.
- **0:55–1:00:** Final review: variable names, off-by-one errors, empty inputs.

TIP: Never start with the hardest problem. Momentum matters. Solving an Easy problem in 15 minutes gives you confidence and time buffer for the Medium.

8.2 When You Are Stuck

If you cannot find the optimal solution in 5 minutes, switch to this survival strategy:

1. Write a **brute-force solution** that is obviously correct. It proves you understand the problem.
2. Add a comment explaining what the optimal approach would be.
3. If time permits, optimize the brute-force by identifying redundant work (caching, early exit, better data structure).

A working brute-force solution scores more than a half-finished optimal one.

NOTE: In the follow-up interview, you can say: "In the timed test I implemented a brute-force $O(n^2)$ solution to ensure correctness. Here is how I would optimize it to $O(n)$..." This demonstrates honesty and deeper understanding.

8.3 Code Quality Under Pressure

Odoo interviewers review your submitted code in the next round. Write code you are proud to explain:

- Use **descriptive variable names** (left, right, current_sum, not i, j, x).
- Add **docstrings** explaining inputs and outputs.
- Handle **edge cases explicitly** (empty string, empty list, single element).
- Avoid one-liners that sacrifice readability.
- Use type hints if comfortable — they signal professionalism.

```
# Good: readable, typed, commented
def has_pair_sum(nums: list[int], target: int) -> bool:
    """Return True if any two distinct elements sum to target."""
    seen = set()
    for num in nums:
        complement = target - num
```



```
        if complement in seen:
            return True
        seen.add(num)
    return False
```

Bad: cryptic one-liner

return any((t-x)in nums for x in nums) # Wrong logic, unreadable

8.4 Environment Checklist

Before clicking Start:

- Test your webcam and lighting. Face the camera directly.
- Close all browser tabs except Coderbyte.
- Disable notifications (OS level).
- Have a physical notepad and pen for sketching.
- Ensure stable internet. If your connection drops, the timer may continue.
- Use the bathroom. 1 hour is a long time when you cannot pause.

8.5 Preparing for the Next Round

If you pass, the technical interview will ask you to:

- Explain your Coderbyte solutions line-by-line.
- Optimize time or space complexity.
- Discuss OOP principles (inheritance, polymorphism, encapsulation).
- Possibly write additional live code.

Prepare by reviewing every solution in this book and asking yourself: "How would I make this faster? What if the input was 10x larger?"

Appendix: Quick Reference Cheat Sheet

Print this page or keep it beside your screen for rapid lookup during practice.

Python String Methods

`s.strip()`, `s.split()`, `s.join()`, `s.replace()`, `s.find()`, `s.count()`, `s.startswith()`, `s.isdigit()`, `s.isalpha()`,
`s[::-1]`

Collections

`from collections import deque, Counter, defaultdict`
`deque`: `appendleft()`, `pop()`, `popleft()`
`Counter`: `most_common()`, `elements()`
`defaultdict(list)`: auto-creates empty lists

SQL Quick Patterns

`SELECT ... FROM ... JOIN ... ON ... WHERE ... GROUP BY ... HAVING ... ORDER BY ...`
`LIMIT`
`INNER JOIN`: matching rows only
`LEFT JOIN`: all left rows, `NULL` for missing right
`RANK()` `OVER (PARTITION BY ... ORDER BY ...)`
Aggregate: `COUNT`, `SUM`, `AVG`, `MAX`, `MIN`

Graph Templates

Adjacency list: `graph = {} ; graph.setdefault(u, []).append(v)`
BFS: `queue = deque([(start, 0)]) ; while queue: node, dist = queue.popleft()`
DFS recursive: `def dfs(node): visited.add(node); for nbr in graph[node]: dfs(nbr)`
DFS iterative: `stack = [start]; while stack: node = stack.pop()`

Stack & Queue

Stack (LIFO): `list.append()`, `list.pop()`, `list[-1]`
Queue (FIFO): `deque.append()`, `deque.popleft()`
Monotonic stack: `while stack and condition: stack.pop()`

Two Pointers

Opposite: `left, right = 0, len(arr)-1 ; while left < right`
Same-direction: `write = 0 ; for read in range(len(arr)): ...`
Sliding window: `window_sum += new - old ; max_sum = max(max_sum, window_sum)`

Math Snippets

Reverse digits: `result = result * 10 + digit`
Prime check: `range(2, int(sqrt(n))+1)`
GCD: `while b: a, b = b, a % b`

Power of two: $n > 0$ and $(n \& (n-1)) == 0$

Big-O Reminders

- Nested loops over n : usually $O(n^2)$
- Single pass with hash set: $O(n)$ time, $O(n)$ space
- Two pointers on sorted array: $O(n)$ time, $O(1)$ space
- BFS/DFS on graph: $O(V + E)$
- Binary search: $O(\log n)$

Good luck. Trust your preparation. Write clean code. Explain your thinking.